
Rapport de stage Stagiaire en réalisation d'application

PELTIER Thomas
Stage du 19/01/2026 au 15/05/2026

BUT Informatique

Maître de stage : Ilandirayane SETTOURAMAN
Enseignant référent : Pierre SANGOUARD

Etablissement : **Ecole Polytechnique**, Route de Saclay, 91120
Palaiseau

Remerciements

Mon stage à l'Ecole Polytechnique étant sur le point de prendre fin, je tiens à remercier toutes les personnes qui m'ont aidé de prêt ou de loin lors de mon projet et dans la rédaction de mon rapport.

Tout d'abord, je tiens à remercier toute l'équipe IDCS de m'avoir permis de travailler au sein de leur unité, de m'avoir accompagné durant ce stage et de leur transmission leur expertise dans le domaine de l'informatique.

Je souhaite également exprimer ma gratitude envers mon maître de stage Ilan SETTOURAMAN pour l'aide qu'il m'a apportée lors des tâches que j'ai eu à réaliser.

Enfin, un grand merci également à Pierre SANGOUARD, mon enseignant référent pour le soutien qu'il m'a donné au cours de mon stage.

Sommaire

Remerciements.....	2
Sommaire.....	3
Introduction générale.....	5
1.Présentation de l'entreprise.....	5
1.1 Présentation de l'Ecole Polytechnique.....	5
1.2 Laboratoire CMLS.....	6
1.3 Laboratoire CPHT.....	7
1.4 Laboratoire CMAP.....	8
1.5 IDCS.....	9
2.Organisation du stage.....	10
2.1 Problématique.....	10
2.2 Objectif du stage.....	10
2.2.1 Fonctionnalités.....	10
3.Missions Initiales.....	11
3.1 Les outils informatiques utilisés.....	11
3.1.1 Git / GitHub.....	11
3.1.2 Framework Svelte.....	11
3.1.3 Langages de programmation.....	11
3.1.4 Editeur de code.....	12
3.2 Phase d'analyse des outils.....	12
3.2.1 GLPI.....	12
3.2.2 Lina.....	12
3.3 Analyse fonctionnelle.....	13
3.4 Analyse détaillée.....	14
3.5 Réalisation des missions.....	16
3.5.1 Créer une base de données.....	16
3.5.2 Récupérer les informations de GLPI et les insérer dans la base de données..	17
3.5.3 Récupérer les informations de Lina et les insérer dans la base de données....	17
3.5.4 Trouver une source exposant les CVE et les insérer dans la base de données...	18
3.5.5 Identifier les machines vulnérables.....	19
3.5.6 Mettre en forme les données sur une page web.....	19
3.5.6.1 Affichage du statut des disques.....	20
3.5.6.2 Affichage des CVE.....	21
3.5.6.3 Affichage des informations des ordinateurs.....	23
3.5.6.4 Affichage des agents Lina.....	24
3.5.6.5 Affichage en détail des CVE.....	25
3.5.7 Proposer une expérience utilisateur (UX) dynamique et adaptable selon les besoins.....	26
3.5.8 Créer une documentation solide pour la maintenance du projet.....	27
3.5.8.1 Mkdocs-material.....	27
3.5.8.2 Swagger.....	27
3.5.9 Créer un système de logs.....	28

4. Missions Supplémentaires.....	29
4.1 Fonctionnalités.....	29
4.2 Phase d'analyse de JAMF.....	29
4.3 Analyse fonctionnelle.....	29
4.4 Analyse détaillée.....	30
4.5 Réalisation des missions.....	31
4.5.1 Modifier la structure de la base de données.....	31
4.5.2 Récupérer les informations de JAMF et les insérer dans la base.....	32
4.5.3 Adapter la page web pour pouvoir changer entre les données de JAMF et celles de GLPI.....	33
4.5.4 Ajouter un composant IA pour faire des requêtes personnalisées à la base de données.....	34
Conclusion.....	35
Bilan professionnel.....	35
Bilan personnel.....	35
Lexique.....	36
Table des figures.....	37
Annexe.....	38

Introduction générale

1. Présentation de l'entreprise

1.1 Présentation de l'Ecole Polytechnique



Figure 1 : Ecole Polytechnique

L'École Polytechnique est une grande école militaire d'ingénieurs créée le 28 septembre 1794 originellement située à Paris. Fidèle à sa devise, "Pour la Patrie, les Sciences et la Gloire", l'école contribue à former des militaires et à créer un espace propice pour la recherche scientifique en France.

C'est dans cette optique qu'elle a été déplacée en 1976 à Palaiseau afin de participer à la création du cluster scientifique et technologique Paris-Saclay. Cela est parfait pour maintenir un niveau d'exigence et de prestige au sein des universités nationales et internationales (1ère place selon l'Étudiant et le Figaro, et 26ème mondiale selon le Times Higher Education).

Il est alors naturel que l'X accueille en son sein de nombreux laboratoires afin de mener des recherches, elle en comptabilise 23 au total qui sont constitués de 606 chercheurs et enseignants chercheurs.

1.2 Laboratoire CMLS

L'un de ces laboratoires est le CMLS (Centre de Mathématiques Laurent SCHWARTZ) dirigé par Nicolas PERRIN. C'est une unité mixte de recherche faisant partie du CNRS (Centre National de la Recherche Scientifique) et de l'école polytechnique.

Ce dernier a plusieurs missions, il est notamment composé en trois équipes de recherche, l'algèbre et l'arithmétique, analyse et équations aux dérivées partielles, géométrie et dynamique. Il participe également à la création des journées mathématiques X-UPS qui ont pour but de former des professeurs de classes préparatoires aux grandes écoles scientifiques.

Organigramme du laboratoire au 1er octobre 2024

Directeur : Nicolas Perrin, Prof. École polytechnique



Figure 2 : Organigramme du CMLS

1.3 Laboratoire CPHT

Il y a également le CPHT (Centre de Physique Théorique), dirigé par Marios PETROPOULOS, actuellement directeur de recherche au CNRS, réunissant des chercheurs couvrant un large spectre de la Physique.

Des séminaires sont régulièrement organisés par les chercheurs pour faire part de leurs recherches et de leurs avancements. Les chercheurs participent également aux enseignements de l'École Polytechnique ainsi qu'à différents types de Masters comme la Physique Théorie, Lasers et Matière et d'autres.

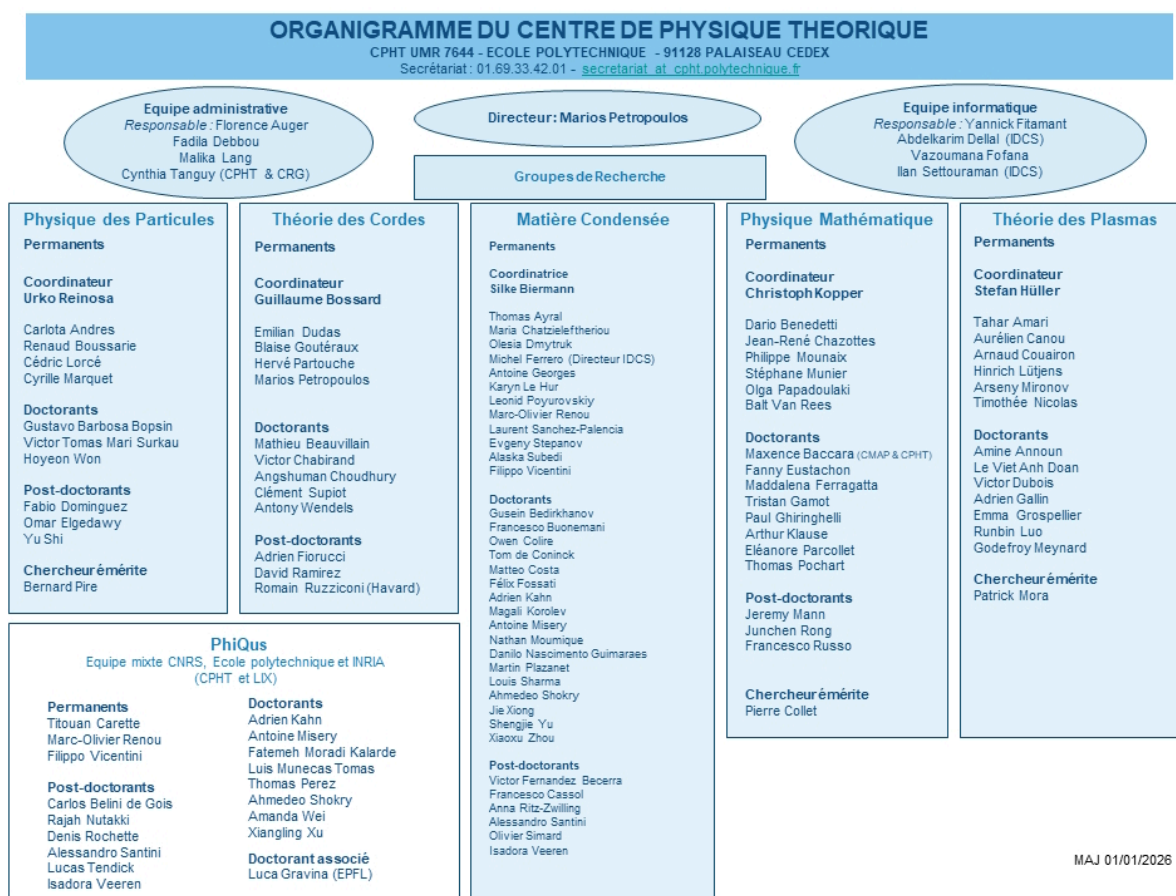


Figure 3 : Organigramme du CPHT

1.4 Laboratoire CMAP

Un autre de ces laboratoires est le CMAP (Centre de Mathématiques Appliquées), sous tutelle principale de l'École Polytechnique et du CNRS, le centre mène des recherches théoriques et numériques sur les mathématiques en interaction avec d'autres sciences (biologie, économie, informatique, mécanique et physique).

Le centre est constitué de plusieurs pôles, le pôle d'analyse, le pôle de probabilités et le pôle décision et données. Le centre est actuellement dirigé par Erwan LE PENNEC assisté dans sa tâche par le directeur adjoint, Michael GOLDMAN. Ils sont également accompagnés d'une équipe administrative prenant en charge la gestion administrative et financière du laboratoire.

Organigramme du Centre de Mathématiques Appliquées

Directeur : Erwan Le Pennec

Directeur Adjoint : Michael Goldman

Informatique :

Pierre Straebler - CMAP

Clément Chanal - CMAP

Pôle analyse :

Anne de Bouard - EDP pour la physique

Grégoire Allaire - Mécanique, Matériaux, Optimisation de Formes

Loïc Gouarin - HPC@Maths

Marc Massot - HPC@Maths

Pietro Congedo - EPC-INRIA PLATON

Pôle probabilités :

Nizar Touzi - Mathématiques Financières

Sylvie Méléard - Population, systèmes particules en interaction

Carl Graham - EPC-INRIA ASCII

Marie Doumic - EPC-INRIA MERGE

Pôle décision et données :

Emmanuel Gobet - Statistiques, apprentissage, simulation, image

Erwan Le Pennec - Statistiques, apprentissage, simulation, image

Anne Auger - EPC RandOpt

Stéphane Gaubert - EPC Tropical

Figure 4 : Organigramme du CMAP

1.5 IDCS

Au total, ces laboratoires comportent 173 chercheurs qui ont besoin de matériels informatiques sur le moment ou qui doit être renouvelé et une garantie de protection des sauvegardes données via des agents extérieurs. Pour cela, l'École Polytechnique a mis en place plusieurs unités pour répondre à ces besoins. IDCS (Infrastructure Données et Calcul Scientifique) est l'une de ces unités, elle assure du support pour CMLS, CPHT, CMAP, CRG, OMEGA avec des informaticiens rattachés à un seul ou plusieurs organismes.

Elle se charge notamment d'approvisionner les laboratoires avec des Macbooks contenant un identifiant unique et un tag représentant l'appartenance au laboratoire CMLS ou CPHT. Les Macbooks sont initialement reçus vierges et IDCS s'occupe de déployer un certain nombre d'agents tournant en tâche de fond sur la machine pour effectuer certaines actions ou renvoyer certaines informations. Il y a notamment l'agent d'inventaire GLPI qui, lorsqu'il est installé sur une machine, va envoyer les informations de la machine au serveur GLPI d'IDCS pour stocker ces informations dans une base de données. Ces informations seront alors disponibles via une interface web qui recense toutes les données présentes dans la base. De même pour l'agent Lina qui, lorsqu'il est installé, se charge d'effectuer une sauvegarde régulière des fichiers et dossiers de l'ordinateur pour pouvoir les restaurer lorsque cela est nécessaire.

Ce sont les éléments principaux concernant le sujet de mon stage, mais le spectre d'activité d'IDCS est beaucoup plus grand. L'unité travaille également avec l'Institut Polytechnique de Paris (IP Paris) en lui fournissant des moyens matériels, logiciels et humains pour faire de la simulation numérique (HPC). L'unité étant également rattachée au cluster de Paris-Saclay, il est commun que des projets avec d'autres institutions voient le jour afin d'en faire bénéficier tout le plateau de Saclay.

2. Organisation du stage

2.1 Problématique

Avec la variété de sujets traités par IDCS, il est impératif que l'unité puisse avoir un outil unique leur permettant de visualiser l'état de leur parc informatique pour être notifié d'une éventuelle brèche de sécurité ou de pouvoir remplacer du matériel obsolète encore en cours d'utilisation. Cependant, les outils actuellement utilisés comme GLPI ou Lina n'offrent pas de solution complètement adaptée aux besoins d'IDCS, d'autant plus qu'il serait nécessaire de consulter ces logiciels un à un afin de récupérer les données essentielles pour effectuer de la maintenance. On peut alors se demander comment mettre en place un système efficace pour consulter les données critiques et les failles de sécurité du parc ?

2.2 Objectif du stage

L'objectif de mon stage était de regrouper les informations fournies par ces outils dans une base de données personnalisée et de mettre à disposition ces données dans une application web. La page web devait être séparée en plusieurs composants, une partie regroupant les informations sur les disques durs des machines, une partie sur les informations de la machine elle-même, une partie sur les potentielles vulnérabilités affectant la machine et une partie regroupant les informations concernant la sauvegarde des données des ordinateurs du parc informatique.

Ce projet demande une interaction entre plusieurs composants logiciels, notamment un programme permettant de récupérer des données extérieures, de pouvoir les stocker et les renouveler dans le temps. Cela couplé avec une communication entre les pages web et la base de données permettra d'avoir une application dynamique et affichant les données en temps réel.

2.2.1 Fonctionnalités

Afin de mener à bien mon projet, j'ai séparé ce qui m'était demandé en plusieurs fonctionnalités distinctes pour avoir une vue d'ensemble de ce que je devais réaliser.

- Créer une base de données
- Récupérer les informations de GLPI et les insérer dans la base de données
- Récupérer les informations de Lina et les insérer dans la base de données
- Trouver une source exposant les CVE et les insérer dans la base de données
- Identifier les machines vulnérables
- Mettre en forme les données sur une page web
- Proposer une expérience utilisateur (UX) dynamique et adaptable selon les besoins
- Créer une documentation solide pour la maintenance du projet
- Créer un système de logs

3. Missions Initiales

3.1 Les outils informatiques utilisés

Afin de faciliter la compréhension des outils et des langages utilisés, je vais faire une brève description de ces derniers.

3.1.1 Git / GitHub

Git est un outil de versionnage permettant d'avoir un dépôt commun où tout le projet est stocké, alors chacun peut ajouter ses fichiers lorsqu'il les a terminé ce qui permet de bien séparer les tâches. Le projet peut également être séparé en branches pour que chaque personne ait son espace de travail et par la suite tout regrouper dans une branche principale lorsque les modifications fonctionnent entre elles.

GitHub utilise Git pour simplifier son utilisation avec une interface web permettant une implémentation beaucoup plus simple et rapide.

J'ai utilisé ces outils afin de revenir à des versions antérieures de mon programme si jamais des modifications que j'essayais d'apporter ne fonctionnaient pas.

3.1.2 Framework Svelte

Svelte est un framework de développement web utilisé pour sa rapidité et simplicité. Cela est dû au fait qu'il compile le code JavaScript à l'avance, ce qui permet de voir les modifications apportées aux pages web en temps réel. Svelte supporte également des technologies nativement comme TypeScript que j'ai utilisées dans mon projet ce qui facilite grandement leur intégration.

3.1.3 Langages de programmation

Go : Go est un langage de programmation développé par Google que j'utilise dans toute la logique de récupération des données. Ce choix a été fait car c'est un langage qui rend le traitement des données extrêmement rapide.

HTML : Il permet de faire les fondations de ce que l'utilisateur verra en brut notamment grâce à l'usage d'éléments et de classes.

CSS : Le CSS est ce qui va embellir le HTML en ajoutant des couleurs, des formes, des styles de polices et la disposition d'éléments sur les pages web.

JavaScript : Afin d'avoir une page dynamique qui puisse interagir avec l'utilisateur. Il est notamment utilisé pour récupérer des données en temps réel ou avoir des fonctionnalités comme des boutons permettant de faire des actions sur les pages.

TypeScript : TypeScript est basé sur JavaScript et a pour but de mieux structurer le JavaScript car il rajoute du typage et sécurise le code pour éviter des potentielles erreurs d'exécution et une meilleure lecture du code.

Shell : Shell est un langage de script qui permet d'effectuer des commandes dans un terminal. Ces commandes peuvent être utilisées pour créer des fichiers, des dossiers ou récupérer des informations sur la machine comme son modèle.

3.1.4 Editeur de code

Visual Studio Code : VS Code est un éditeur de code développé par Microsoft qui contient plein de fonctionnalités notamment, le débogage, la mise en évidence des erreurs de syntaxe dans le code et une intégration Git directement dans le logiciel. Il est également customisable à souhait selon les extensions téléchargeables mises à disposition dans l'application ce qui m'a permis d'adapter mon environnement avec ce dont j'avais besoin comme une extension pour Go et Svelte.

3.2 Phase d'analyse des outils

3.2.1 GLPI

GLPI (Gestionnaire Libre de Parc Informatique) est un agent installé sur les ordinateurs où il effectue un scan matériel et logiciel pour en récupérer diverses informations tels que le modèle, son utilisateur, son système d'exploitation et les logiciels installés. Ces scans sont ensuite envoyés vers le serveur GLPI pour alimenter la base de données de l'inventaire du parc informatique.

Il m'est possible de récupérer les informations de la base de données grâce à une API à qui il faut envoyer une requête afin que le serveur me renvoie une réponse contenant ces informations. Cependant, il est nécessaire de s'authentifier lors de la requête avec un token créé pour cette tâche, cela évite que n'importe quel utilisateur faisant un appel puisse récupérer des informations sensibles.

3.2.2 Lina

Lina est un produit de l'entreprise Atempo, son agent détecte en temps réel les données nouvelles et modifiées sur le poste de travail, puis applique les politiques de protection définies par l'administrateur et l'utilisateur. La protection s'effectue de manière automatique et continue, sans programmation de sauvegarde, avec une fréquence d'envoi définie par la Perte de Données Maximale Admissible (PDMA) qui peut varier d'une minute à un jour. L'agent offre également aux utilisateurs une capacité de restauration en libre-service via une interface Web moderne, sans nécessiter l'intervention de l'administrateur.

Lina met également à disposition une interface web contenant un tableau de bord, mais surtout des informations sur les agents présents sur les machines. Comme GLPI, Lina a une API qui, lorsque interrogée, retourne les informations des agents comme leur dernière date de sauvegarde pour voir si cette dernière a eu lieu il y a trop de temps, mais aussi la version de l'agent lui-même afin de vérifier si l'agent a bien la même version que le serveur Lina. Si cette version est différente, l'agent peut alors se mettre à jour automatiquement.

3.3 Analyse fonctionnelle

Après avoir pris connaissance de ce que je devais faire et des outils à utiliser, j'ai réalisé une courte description afin que les attentes de mon maître de stage correspondent à ce que je vais développer.

Fonctionnalité 1 : Créer une base de données

Mon maître de stage m'a donné une liste des informations qu'il souhaite avoir sur le tableau de bord, à partir de cette liste j'ai créé un schéma afin que celui-ci soit conforme à sa demande.

Fonctionnalité 2 : Récupérer les informations de GLPI et les insérer dans la base de données.

Récupérer les informations concernant les ordinateurs, les disques durs, les systèmes d'exploitation et les logiciels installés grâce à l'API de GLPI, puis les insérer dans la base de données personnelle du projet. Il est également nécessaire de vérifier les informations récupérées à chaque appel, en vue d'une éventuelle mise à jour des données.

Fonctionnalité 3 : Récupérer les informations de Lina et les insérer dans la base de données.

Récupérer les informations concernant les agents Lina installés sur les machines via l'API de Lina puis les insérer dans la base de données personnelle du projet. De même que pour GLPI, il est nécessaire de vérifier les informations récupérées afin de faire une mise à jour dans la base de données s'il le faut.

Fonctionnalité 4 : Trouver une source exposant les CVE et les insérer dans la base de données.

Une CVE est une vulnérabilité informatique pouvant concerner un système d'exploitation ou un logiciel, ces dernières ont un score allant de 0 à 10 selon le niveau de gravité de la CVE et à quel point elle est exploitable. Dans le pire des cas, une vulnérabilité considérée notée à 10 peut permettre à l'attaquant de lancer directement des programmes sur l'ordinateur où le système d'exploitation / logiciel est installé. Les CVE sont composées d'une ou plusieurs CPE qui indiquent précisément quel domaine est affecté car une CVE peut affecter plusieurs logiciels et systèmes d'exploitation simultanément. Les CPE peuvent aussi contenir la version à laquelle la vulnérabilité a été réparée par l'entreprise du système d'exploitation / du logiciel.

Afin de récupérer les données sur les vulnérabilités, j'ai décidé d'utiliser la base de données NVD (National Vulnerability Database) avec son API afin de lui faire des requêtes

pour récupérer les vulnérabilités des systèmes d'exploitation et des logiciels. Je les insérerai par la suite dans la base de données.

Fonctionnalité 5 : Identifier les machines vulnérables

Avec les informations disponibles dans ma base de données, je dois récupérer les informations des ordinateurs et les comparer avec les vulnérabilités récupérées pour savoir si ces derniers sont à risques ou non et mettre à jour l'information dans la base si c'est le cas.

Fonctionnalité 6 : Mettre en forme les données sur une page web

Faire une API qui expose les données sur une URL pour permettre à mes pages web de faire un appel à cet URL et récupérer les données pour les afficher sur une page web.

Fonctionnalité 7 : Proposer une expérience utilisateur (UX) dynamique et adaptable selon les besoins.

Étant donné que la page web sera composée de plusieurs rectangles d'informations, j'ai proposé de pouvoir interchanger les rectangles de position et d'afficher ou non certains rectangles pour permettre à l'utilisateur de se faire son propre tableau de bord customisable.

Fonctionnalité 8 : Créer une documentation solide pour la maintenance du projet.

Fournir une documentation détaillée permettant de maintenir le projet à jour selon les mises à jour des différentes données.

Fonctionnalité 9 : Créer un système de logs.

Ajouter un système de logs dans tout le backend qui écrit les informations dans un fichier pour vérifier la bonne exécution du programme.

3.4 Analyse détaillée

Fonctionnalité 1 : Créer une base de données

Création de multiples tables, notamment une table pour stocker les informations des ordinateurs de GLPI, une table pour stocker les informations des agents Lina et une table pour stocker les informations à propos des CVE.

Fonctionnalité 2 : Récupérer les informations de GLPI et les insérer dans la base de données.

Création d'une fonction pour créer un client GLPI pour pouvoir être authentifié lors des requêtes de récupération de données à l'API de GLPI.

Création d'une fonction récupérant les informations des ordinateurs grâce à l'API de GLPI en utilisant le client.

Création d'une fonction insérant les informations des ordinateurs dans la base avec une vérification de l'existence de l'ordinateur pour le mettre à jour si nécessaire.

Fonctionnalité 3 : Récupérer les informations de Lina et les insérer dans la base de données.

Création d'une fonction récupérant les informations des agents Lina en utilisant mon token API.

Création d'une fonction pour insérer les informations des agents Lina dans la base avec une vérification de l'existence de l'ordinateur pour le mettre à jour si nécessaire.

Fonctionnalité 4 : Trouver une source exposant les CVE et les insérer dans la base de données.

Création d'une fonction récupérant les informations des vulnérabilités de la base de données NVD en utilisant mon token API.

Création d'une fonction pour insérer les informations des vulnérabilités dans la base de données.

Fonctionnalité 5 : Identifier les machines vulnérables.

Création d'une fonction qui récupère toutes les vulnérabilités et vérifie si les systèmes d'exploitation ou les logiciels sont impactés par la vulnérabilité et insère l'information dans un champ de la base de données.

Création d'une fonction qui va compter le nombre de machines affectées par la vulnérabilité et ajouter la donnée dans une table prévue à cet effet.

Fonctionnalité 6 : Mettre en forme les données sur une page web.

Création d'un routeur qui met à disposition des URL en local dans lesquelles je vais mettre à disposition les données de ma base pour créer un système d'API.

Création de pages web interrogeant les URL mis en place par mon API afin de récupérer les données et les afficher sur les pages web.

Fonctionnalité 7 : Proposer une expérience utilisateur (UX) dynamique et adaptable selon ses besoins.

Mise en place d'une Grid qui récupère chaque composant affichant des informations et les dispose sur la page.

Création d'une sidebar rétractable qui permet de choisir les différents composants à afficher.

Création d'une fonction permettant de drag and drop les composants de la page pour permettre à l'utilisateur de la personnaliser.

Fonctionnalité 8 : Créer une documentation solide pour la maintenance du projet.

Utilisation de MkDocs pour la documentation avec plusieurs rubriques expliquant comment installer le projet, son architecture et la rubrique maintenance. Cette rubrique étant la plus importante.

Utilisation de Swagger générant la documentation de mon API couplée avec MkDocs permettant l'insertion de ma documentation Swagger dans MkDocs.

Fonctionnalité 9 : Créer un système de logs.

Création d'un logger avec la bibliothèque ZapLogger et utilisation de ce logger pour afficher tous les messages d'erreurs potentiels et les messages de bonnes exécutions des tâches dans un fichier de logs.

3.5 Réalisation des missions

3.5.1 Créer une base de données

Afin d'avoir une base solide sur laquelle travailler, j'ai commencé par créer les quatre tables principales, « computer_glpi_dbs », « agent_lina_dbs », « cvedbs » et « cpe_dbs » pour mettre en forme les informations qui doivent être stockées. Par la suite, je me suis occupé de faire les liens entre les CVE, les logiciels « installed_software_glpi » et les systèmes d'exploitation « operating_system_glpi » afin d'identifier quels logiciels et quels systèmes sont vulnérables et stocker cette information dans les deux tables prévues à cet effet. Ensuite, je peux stocker le nombre de machines impactées en récupérant les données de mes tables pour renseigner cette information dans le champ « machines_impacted » de la table « cvedbs ».

Quant à la table « sync_dates », elle me servira à savoir quand mon programme s'est exécuté pour la dernière fois. Grâce à cela, je peux optimiser la manière dont je récupère mes données pour ne pas rajouter du temps de calcul lors de l'exécution de mon programme.

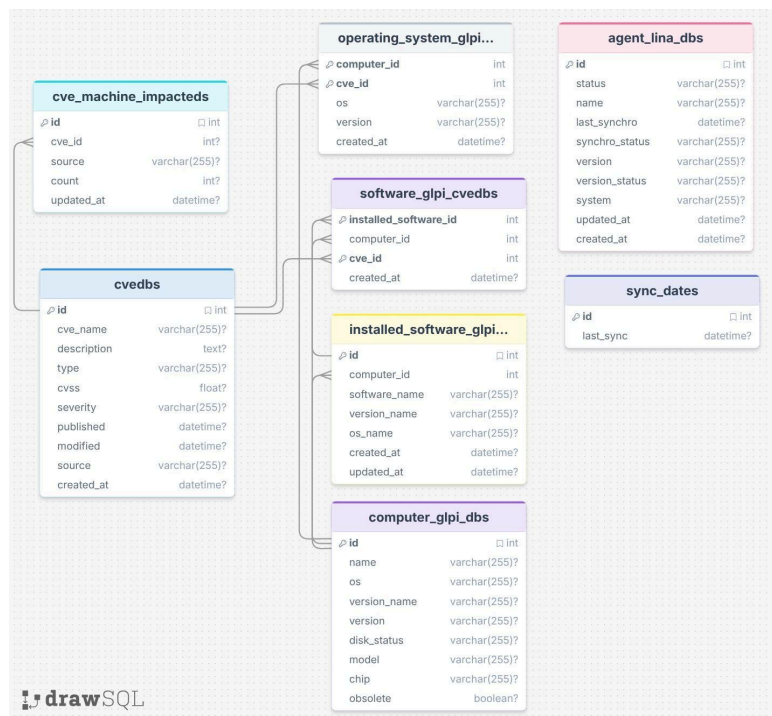


Figure 5 : Schéma relationnel de la base de données

3.5.2 Récupérer les informations de GLPI et les insérer dans la base de données

La plupart des APIs requièrent un moyen d'authentifier la provenance de la requête car on touche à des données assez sensibles la plupart du temps. Pour pallier ce problème, il a déjà fallu inspecter GLPI afin de voir comment celui-ci permettait de faire des requêtes API. La documentation du logiciel ne donnant pas beaucoup d'informations et le forum étant plutôt inactif, nous avons avec mon maître de stage trouvé une solution pour me créer un Token m'identifiant et à utiliser lors de mes requêtes.

Après avoir réussi à faire un appel API pour récupérer des données, j'ai rapidement constaté que la manière dont les données sont structurées chez GLPI est très peu intuitive. Par exemple, si je souhaite récupérer les informations concernant un ordinateur dans GLPI, au lieu de me renvoyer directement le nom du système d'exploitation, GLPI va me renvoyer un identifiant correspondant au système d'exploitation. Il faut ensuite refaire un appel API pour voir à quoi cet identifiant correspond pour enfin récupérer le nom du système d'exploitation. Souhaitant récupérer les informations de l'ordinateur, son système d'exploitation avec version et ses logiciels avec version, il est nécessaire de faire 5 appels différents en attendant que le premier se termine, puis regrouper les informations soi-même, en fonction des identifiants que la machine possède ce qui n'est pas optimal. Pour optimiser au maximum l'exécution du programme, j'ai séparé les différents appels en ce que l'on appelle des « go routine ». Cela sépare l'exécution du programme en plusieurs parties afin de ne pas attendre que les appels API se finissent un à un mais de faire tout en même temps pour ensuite regrouper les données de l'ordinateur.

Malheureusement, les informations que propose l'API de GLPI restent limitées, il m'est impossible de récupérer le statut des disques pour savoir s'ils sont chiffrés ou non. Pour pallier ce problème, j'ai dû trouver une solution extérieure pour me permettre de récupérer ces informations.

Tout d'abord, j'ai programmé des scripts shell, qui en s'exécutant, lancent des commandes renvoyant des informations concernant la machine sur laquelle le script est lancé. Etant donné que GLPI a la capacité de déployer et d'exécuter des scripts sur toutes les machines, il a fallu créer des groupes de machines afin de ne pas cibler celles qui ne sont pas du CMLS ou du CPHT. Ces tâches n'étant exécutables que depuis un navigateur, j'ai utilisé Playwright qui simule un navigateur permettant d'aller sur des pages et de cliquer aux endroits définis dans mon programme. Avec cette combinaison d'outils, j'ai pu lancer des tâches en cliquant sur les boutons du navigateur, puis récupérer les informations des scripts en exportant un fichier CSV dans mon programme pour en tirer les données à insérer dans ma base de données.

3.5.3 Récupérer les informations de Lina et les insérer dans la base de données

La manière de récupérer des informations depuis Lina est la même que celle de GLPI, il faut s'authentifier à l'aide d'un token qui définit les différents droits que l'utilisateur

possède ce qui lui permet de récupérer ou non les informations de Lina. Après avoir épluché la documentation, je me suis rendu compte que le seul moyen d'avoir accès aux informations dont j'avais besoin était d'avoir un token avec les droits de superadmin. Cependant, pour des raisons de sécurité, IDCS ne peut pas me permettre d'avoir un compte avec ces privilèges. Afin d'avoir plus d'informations, j'ai pris contact par mail avec Mr. Laurent Charmet, développeur chez Lina qui m'a confirmé qu'il n'y avait pas d'autres possibilités pour récupérer les données depuis Lina.

Pour résoudre ce problème, je me suis encore tourné vers Playwright m'offrant la possibilité d'aller sur le site et d'exporter un fichier CSV. En lisant le fichier avec mon programme, j'ai pu en extraire les informations dont j'avais besoin pour ma base de données et je les ai insérées dedans. Toutefois, certaines informations étaient manquantes, notamment l'information me disant si le Lina de la machine était à jour et si sa dernière synchronisation a été faite récemment. J'ai alors mis en place une procédure de récupération de la version de Lina du serveur pour la comparer avec celle de tous les agents et mis une vérification de 3 jours sur la date de synchronisation pour me permettre de savoir si l'agent doit en effectuer une.

A l'heure actuelle, IDCS est toujours en phase de migration de Lina sur un autre serveur, j'ai donc adapté mon programme afin qu'il puisse récupérer les informations de l'ancien serveur et les informations du nouveau serveur.

3.5.4 Trouver une source exposant les CVE et les insérer dans la base de données.

Avec les données des ordinateurs stockées dans ma base, il fallait trouver un moyen de récupérer les CVE pouvant toucher les systèmes d'exploitation ou logiciels des ordinateurs. Pour cela, j'ai utilisé le site du NIST (National Institute of Standards and Technology) qui a une branche NVD (National Vulnerability Database) offrant une API que je peux utiliser afin de récupérer toutes les CVE renseignées. Après avoir créé un token identifiant les requêtes comme appartenant à l'organisme IDCS, j'ai pu commencer à faire des requêtes.

Cependant, la requête pour récupérer les données prend beaucoup de temps à s'exécuter dû au nombre de CVE et CPE présentes dans la base de données NVD, environ 300 000 CVE et des millions de CPE.

Afin d'optimiser le temps de calcul, j'ai ajouté des filtres sur la requête et dans le tri des données pour ne récupérer que les CVE représentant un réel danger de sécurité. De plus, je stocke dans ma base de données la date de la dernière exécution du programme, cela me permet d'adapter la requête lors de la prochaine exécution pour récupérer seulement les CVE qui ont été modifiées récemment.

3.5.5 Identifier les machines vulnérables

Maintenant que toutes les CVE dangereuses ont été stockées dans la base de données, il fallait mettre en place le fait de pouvoir identifier celles qui impactent vraiment le parc. C'est important car dans chaque CPE, il peut y avoir une indication disant à quelle version la vulnérabilité est fixée. Les systèmes d'exploitation et logiciels des ordinateurs ayant tous une version différente, il est important de comparer ces versions pour éviter de fausser les données concernant des machines qui ne sont pas réellement affectées.

Afin de régler ce problème, j'ai mis en place un programme récupérant tous les systèmes d'exploitation des ordinateurs et logiciels stockés dans ma base pour les comparer avec chaque CPE qui pouvait potentiellement les affecter. Cependant, ce processus prenait encore trop de temps, j'ai alors dû m'adapter en triant les données que je récupérais, notamment filtrer les CPE par leurs types pour savoir si elles affectent les systèmes d'exploitation ou les logiciels pour optimiser le temps de calcul.

3.5.6 Mettre en forme les données sur une page web

Les informations étant stockées dans la base de données, j'ai mis en place une API locale en utilisant le framework web Gin. Grâce à cela, je peux exposer mes données sur des endpoints HTTP en format JSON afin qu'elles puissent être exploitées par mon application web.

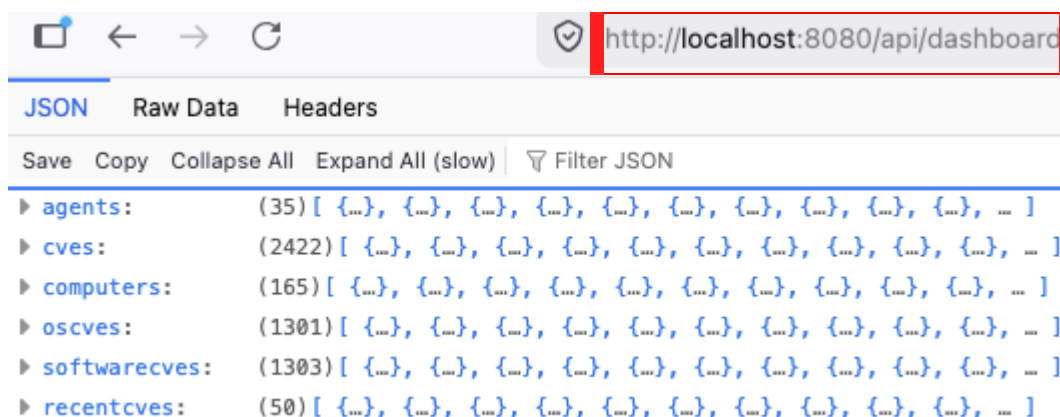


Figure 6 : Capture d'écran de l'API

Avec la capture d'écran ci-dessus, je peux voir quelles données sont envoyées par mon programme sur l'URL, (encadré rouge) de mon API. Avec ce système, je peux faire une requête fetch depuis une page web qui consiste à récupérer les objets qui sont présents sur cette URL et les afficher sur la page.

Pour mettre en forme les données, j'ai décidé de transformer la page principale en une grille de composants, de cette manière cela permet de bien séparer les informations pour un affichage clair et épuré.

3.5.6.1 Affichage du statut des disques

Afin d'avoir un aperçu clair des informations, j'ai fait un diagramme circulaire avec une légende reprenant les trois statuts possibles d'un disque, « OK », « BAD_KEY » et « DISABLED ». Le statut BAD_KEY est très important car les Mac récents sont déjà chiffrés mais pas forcément avec la bonne clé d'entreprise, il faut alors modifier pour qu'elle soit conforme au laboratoire de l'ordinateur étant donné que la clé d'entreprise n'est pas la même entre le CPHT et le CMLS.

J'ai également ajouté un tableau ci-dessous pour avoir une vue plus globale sur les disques des ordinateurs permettant de voir directement lesquels sont impactés.

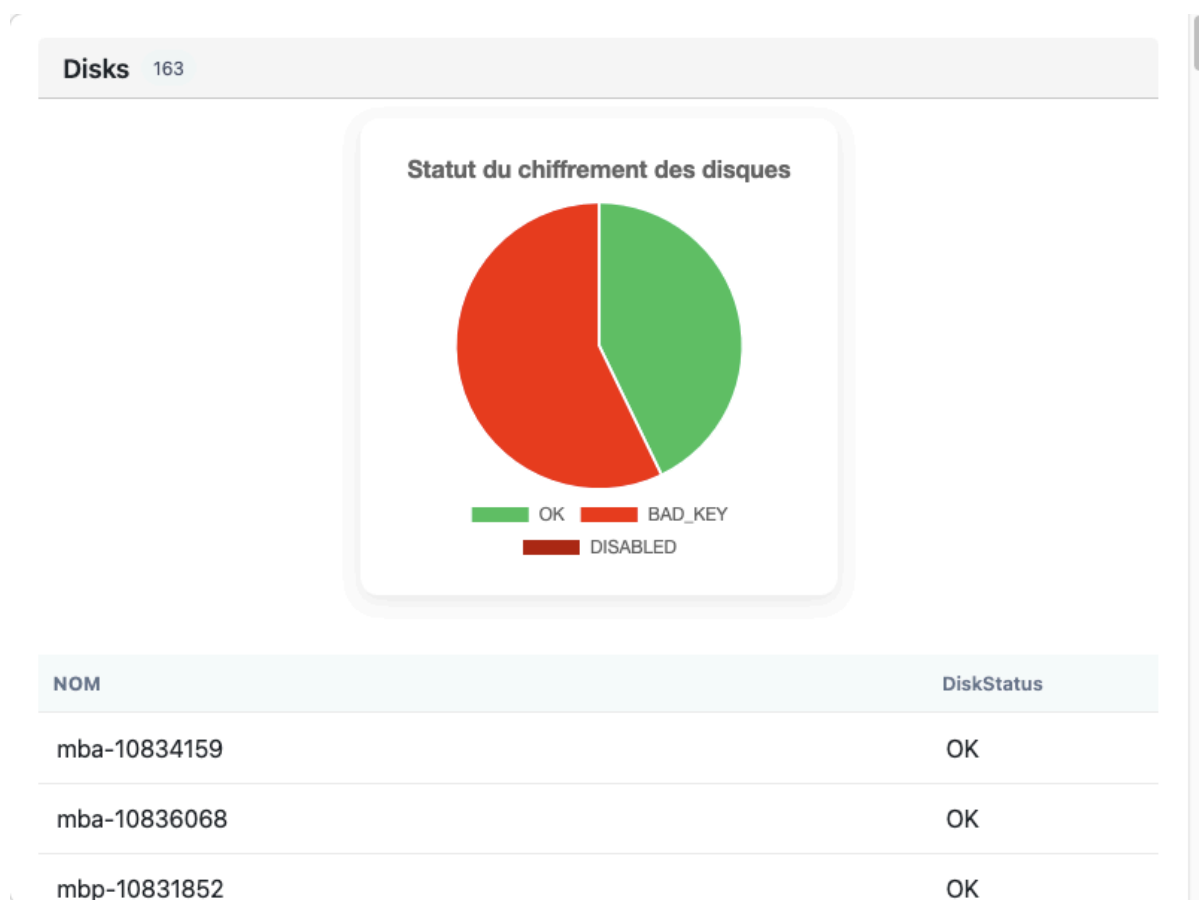
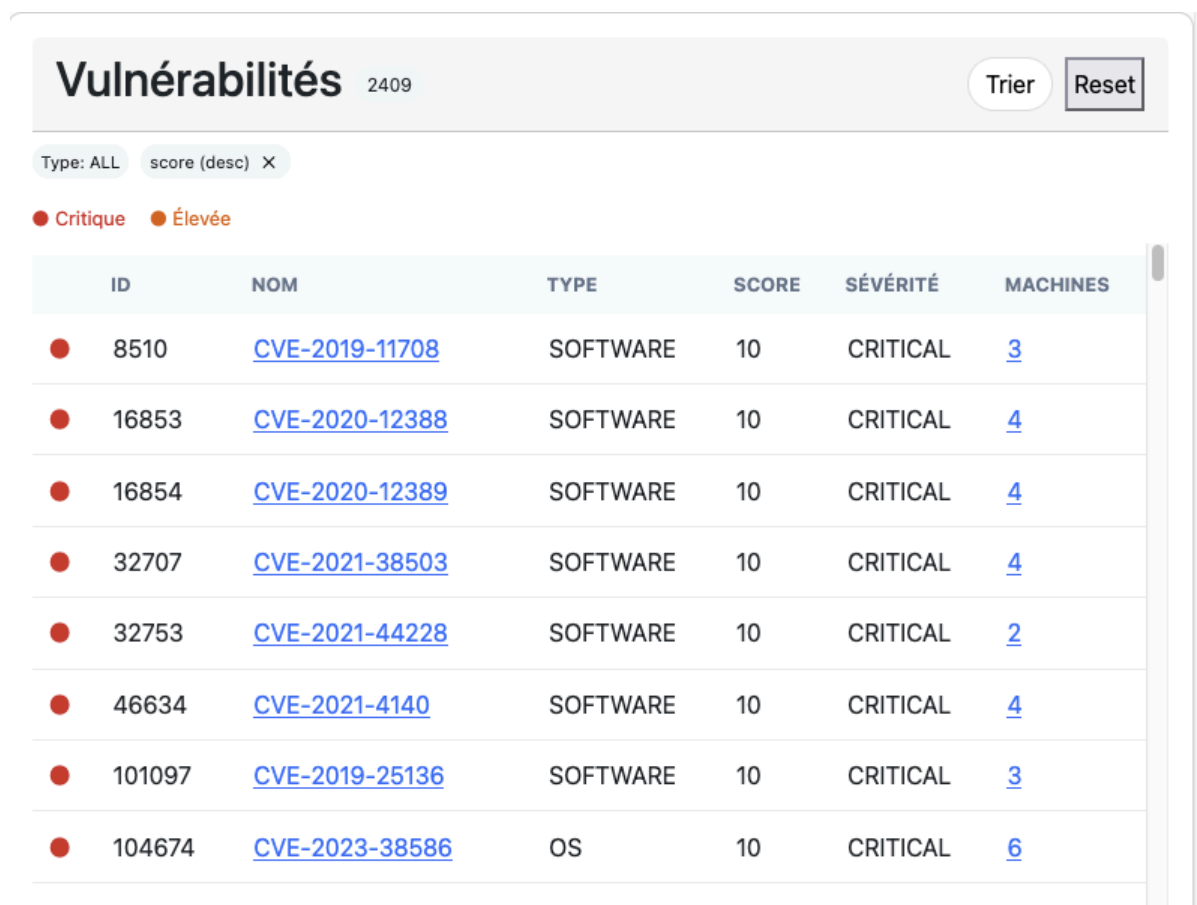


Figure 7 : Composant du statut des disques

3.5.6.2 Affichage des CVE

L'affichage pour les CVE est plutôt différent car il y en a plus de deux milliers, j'ai tout de même gardé l'approche de faire un tableau pour structurer les données car cela me paraît être la plus efficace. Cependant, il est impératif de mettre en place une manière de trier les données sinon le tableau serait inutilisable, alors j'ai programmé la possibilité de trier en multi critères lorsque l'on clique sur le bouton « Trier » où l'option de trier par type, par score et par nombre de machines affectées s'affichera, et chaque critère sera affiché au-dessus du tableau afin de montrer lequel est actif.



ID	NOM	TYPE	SCORE	SÉVÉRITÉ	MACHINES
● 8510	CVE-2019-11708	SOFTWARE	10	CRITICAL	3
● 16853	CVE-2020-12388	SOFTWARE	10	CRITICAL	4
● 16854	CVE-2020-12389	SOFTWARE	10	CRITICAL	4
● 32707	CVE-2021-38503	SOFTWARE	10	CRITICAL	4
● 32753	CVE-2021-44228	SOFTWARE	10	CRITICAL	2
● 46634	CVE-2021-4140	SOFTWARE	10	CRITICAL	4
● 101097	CVE-2019-25136	SOFTWARE	10	CRITICAL	3
● 104674	CVE-2023-38586	OS	10	CRITICAL	6
● 104700	CVE-2023-10155	OS	10	CRITICAL	4

Figure 8 : Composant des CVE

J'ai également ajouté des liens, le premier sur le nom de la CVE emmenant directement sur le site officiel de NVD concernant la dite CVE pour avoir plus d'informations la concernant. Toutefois, si l'utilisateur souhaite rester sur le dashboard, il peut aussi cliquer sur le nombre de machines pour avoir une courte description de la CVE et voir quelles machines sont affectées précisément.

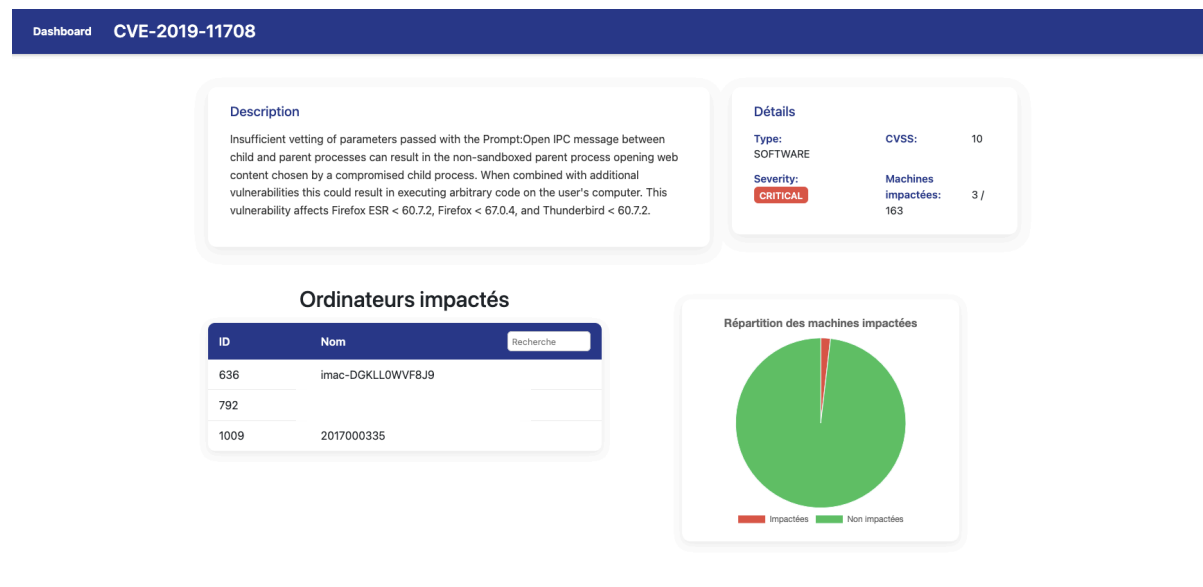


Figure 9 : Page web des détails de la CVE

3.5.6.3 Affichage des informations des ordinateurs

Le système d'exploitation des Mac fonctionne par version, si un Mac est trop vieux et qu'il est sous une version non supportée par Apple il est nécessaire de le changer car celui-ci devient obsolète. Pour mettre ça en forme visuellement j'ai décidé de faire un graphique en barre illustrant la proportion de Mac obsolètes et non obsolètes. En dessous, j'ai ajouté un tableau affichant le système d'exploitation, la version du système et d'autres informations importantes à avoir sur le dashboard concernant les ordinateurs.



Figure 10 : Composant des informations des ordinateurs

3.5.6.4 Affichage des agents Lina

Pour mettre en forme les données des agents Lina, j'ai choisi de faire deux diagrammes circulaires, le premier consistant à représenter la part des agents qui ont fait leur synchronisation récemment, le deuxième pour voir si les agents Lina ont la même version que le serveur Lina. En dessous, il y a encore un tableau pour afficher les données et pouvoir les trier selon plusieurs critères comme la date de synchronisation ou le statut de la version, ainsi que la possibilité de rechercher un agent par son nom.

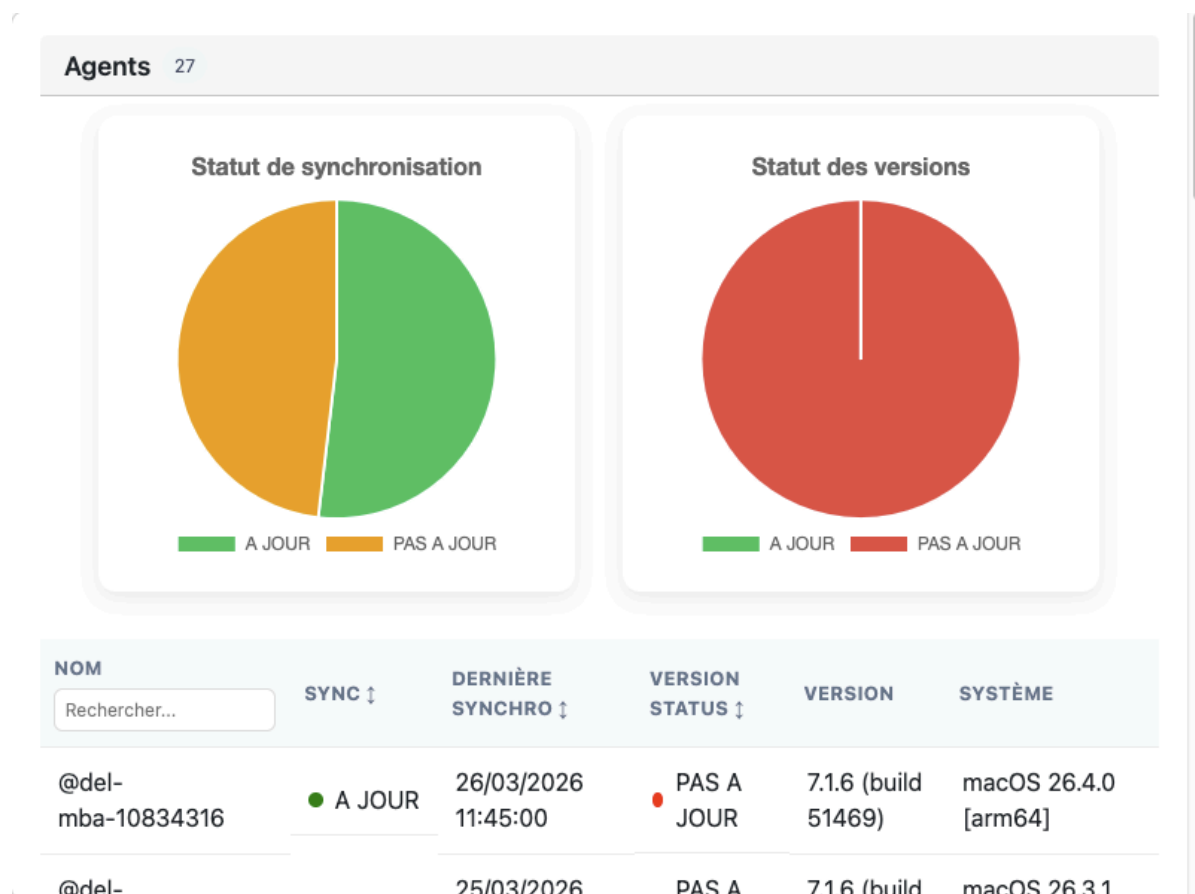


Figure 11 : Composant des agents Lina

3.5.6.5 Affichage en détail des CVE

J'ai proposé à mon tuteur de faire un composant plus grand sur les CVE contenant 3 tableaux différents pour avoir une vue plus globale. Les deux premiers tableaux auront chacun un type de CVE, un pour les systèmes d'exploitation et un pour les logiciels tandis que le dernier sera pour afficher les CVE les plus récentes.

CVEs Détails

● Critique ● Élevée

OS CVEs

Trier

score (desc) X

ID	NOM	TYPE	SCORE	SÉVÉRITÉ	MACHINES
● 104674	CVE-2023-38586	OS	10	CRITICAL	6
● 104703	CVE-2023-40455	OS	10	CRITICAL	6
● 120195	CVE-2024-44146	OS	10	CRITICAL	18
● 120196	CVE-2024-44148	OS	10	CRITICAL	18
● 127007	CVE-2025-24085	OS	10	CRITICAL	29
● 129237	CVE-2025-24201	MIXED	10	CRITICAL	30
● 137857	CVE-2025-43300	OS	10	CRITICAL	52

Software CVEs

Trier

score (desc) X

ID	NOM	TYPE	SCORE	SÉVÉRITÉ	MACHINES
● 8510	CVE-2019-11708	SOFTWARE	10	CRITICAL	3
● 16853	CVE-2020-12388	SOFTWARE	10	CRITICAL	4
● 16854	CVE-2020-12389	SOFTWARE	10	CRITICAL	4
● 32707	CVE-2021-38503	SOFTWARE	10	CRITICAL	4
● 32753	CVE-2021-44228	SOFTWARE	10	CRITICAL	2
● 46634	CVE-2021-4140	SOFTWARE	10	CRITICAL	4
● 101097	CVE-2019-25136	SOFTWARE	10	CRITICAL	3

Recent CVEs

Trier

date (desc) X

ID	NOM	TYPE	SCORE	SÉVÉRITÉ	MACHINES	DATE
● 250880	CVE-2026-26133	SOFTWARE	7.1	HIGH	1	09/04/2026
● 254217	CVE-2026-34769	SOFTWARE	7.7	HIGH	1	09/04/2026
● 255749	CVE-2026-5735	SOFTWARE	9.8	CRITICAL	108	07/04/2026
● 255748	CVE-2026-5734	SOFTWARE	9.8	CRITICAL	108	07/04/2026
● 255747	CVE-2026-5733	SOFTWARE	8.8	HIGH	108	07/04/2026
● 255746	CVE-2026-5732	SOFTWARE	8.8	HIGH	108	07/04/2026
● 253934	CVE-2026-22561	SOFTWARE	7.8	HIGH	2	06/04/2026

Figure 12 : Composant des détails des CVE

La même logique de tri des CVE et le fait d'aller sur la page permettant d'avoir une description de ces dernières s'appliquent également à ce composant pour faciliter l'utilisation de la page.

3.5.7 Proposer une expérience utilisateur (UX) dynamique et adaptable selon les besoins

J'ai proposé l'idée de pouvoir afficher les différents composants selon les préférences de l'utilisateur. Grâce à la séparation de la page web en plusieurs composants distincts, il est simple d'effectuer une liste des composants que l'utilisateur souhaite avoir. Pour voir cette liste, j'ai conclu qu'un bouton la faisant apparaître serait la meilleure option pour ne pas rajouter trop d'éléments visuels à un dashboard déjà bien complet. Cette liste prend la forme des noms des composants avec un slider qui permet de les sélectionner ou de les désélectionner en cliquant dessus. J'ai également rajouté un fond qui s'obscurcit et floute la page pour indiquer à l'utilisateur qu'il est bien en train de sélectionner les composants qu'il souhaite avoir.

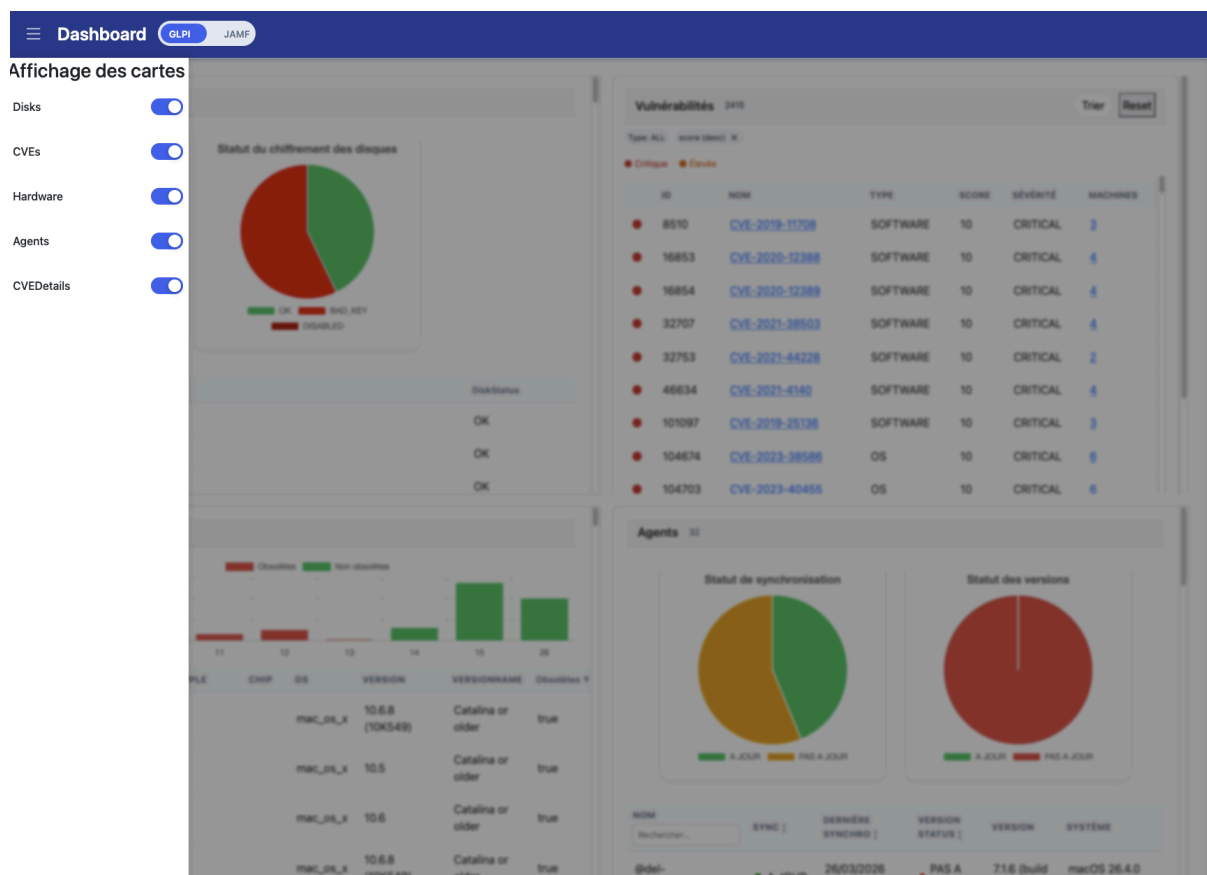


Figure 13 : Affichage de la liste des composants

J'ai également utilisé la bibliothèque « svelte-dnd-action » pour implémenter du drag and drop sur l'intégralité des composants. De cette manière, il est possible pour l'utilisateur de faire son propre dashboard en restant cliquer sur les composants longtemps et de les déplacer là où il veut.

3.5.8 Créer une documentation solide pour la maintenance du projet

3.5.8.1 Mkdocs-material

Le projet devenant lourd et utilisant de multiples outils, il est devenu important de créer une documentation robuste afin qu'il ne soit pas inutilisable par la suite si un problème de maintenance venait à arriver. J'ai choisi d'utiliser mkdocs-material pour réaliser la documentation car j'ai déjà de l'expérience avec cet outil, d'autant plus qu'il est parfaitement adapté pour créer une documentation avec plusieurs rubriques et une barre de recherche intégrée en utilisant des fichiers Markdown ce qui simplifie beaucoup la création de la documentation.

Cela répond à tous mes besoins étant donné que certaines solutions que j'utilise actuellement pour récupérer des informations, notamment les exports de fichiers CSV Lina sont un moyen de contourner le processus de base qui est d'utiliser son API. Selon moi, il est primordial que tous ces moyens de contournement qui fonctionnent pour l'instant mais ne sont pas réellement du code propre soient bien documentés dans une rubrique « Maintenance ». Il sera alors simple de pouvoir modifier la documentation si jamais quelque chose vient à changer dans le fonctionnement du projet.

3.5.8.2 Swagger

Mkdocs-material reste un très bon outil pour de la documentation générale mais il en existe d'autres qui sont spécialisés dans certains domaines, notamment Swagger. Swagger se spécialise dans la réalisation de documentation d'API en générant automatiquement une documentation à partir de commentaires laissés dans mes fichiers Go.

Son utilisation offre beaucoup d'avantages comme la possibilité d'avoir une documentation interactive et dynamique en une seule commande et de voir les objets que mon API renvoie par exemple les ordinateurs et les champs contenant leurs informations. Il est également possible de faire des appels API depuis la documentation elle-même pour vérifier si tout fonctionne. Toutes ces fonctionnalités couplées au fait que je puisse importer directement les fichiers générés par Swagger dans mes fichiers markdown réalisés avec Mkdocs-material me permettent d'avoir une documentation solide et au même endroit afin qu'elle corresponde au mieux selon aux besoins d'IDCS.

3.5.9 Créer un système de logs

Pour avoir un projet robuste il est nécessaire d'avoir une bonne documentation mais aussi d'avoir un bon système de logs afin de déterminer un problème d'exécution rapidement si cela venait à se produire. C'est d'autant plus important dans mon projet qui utilise beaucoup de données extérieures venant de plusieurs points différents, (GLPI, Lina, NVD). Il m'est souvent arrivé de voir des échecs lors de l'exécution de mon programme sans réellement comprendre d'où en venait la source.

Afin de résoudre ce problème j'ai utilisé une librairie qui s'appelle Zaplogger, cette dernière a beaucoup d'avantages car elle me permet d'écrire dans un fichier .log toutes les informations que je souhaite avec un certain niveau de criticité. Je peux alors l'utiliser pour écrire lorsqu'il y a une erreur lors de mon programme en utilisant le niveau ERROR mais aussi quand une exécution s'est bien passée avec INFO. Il est même possible de rajouter des informations lors de l'exécution des messages, grâce à cela j'ai documenté chaque exécution importante de mon programme ce qui me permet de savoir quelles parties de mon projet s'exécutent et lesquelles peuvent planter et pourquoi. En plus de cela, Zaplogger s'occupe automatiquement de définir la taille maximum du fichier car toutes ces lignes stockées dans un fichier peuvent le rendre lourd. Zaplogger permet également d'avoir un nombre défini de backups du fichier et de régler l'âge maximum du fichier de log pour savoir s'il doit être remplacé.

Grâce à cela, j'ai un fichier « app.log » avec toutes les informations de mon programme regroupées en un seul endroit, consultable avec un outil d'analyse de logs. Ces outils permettent de lire ces fichiers .log car l'information est écrite en brut dans le fichier ce qui n'est pas adapté pour l'œil humain.

4. Missions Supplémentaires

Suite à la finalisation de mon projet, nous avons remarqué que GLPI ne correspondait pas aux besoins de ressourcement d'informations du parc informatique. L'idée de faire une phase de transition vers un autre agent d'inventaire était déjà présente mais mon projet a conforté l'unité de faire la transition vers un nouvel agent appelé JAMF.

La transition n'étant pas encore terminée, nous en avons conclu que la meilleure solution était de pouvoir choisir d'afficher les données concernant JAMF ou GLPI directement depuis le dashboard. De cette manière, lorsque la transition sera terminée, il suffira de couper la partie GLPI du programme et toute la partie JAMF restera intacte. Cela rendra le projet flexible car il sera toujours possible de revenir vers GLPI plus tard si nécessaire ou bien d'ajouter un nouvel outil de manière individuelle.

4.1 Fonctionnalités

De la même manière que pour mon projet initial, j'ai séparé la demande de mon tuteur en plusieurs fonctionnalités afin d'avoir une idée claire sur comment réaliser mes missions.

- Modifier la structure de la base de données
- Récupérer les informations de JAMF et les insérer dans la base
- Adapter la page web pour pouvoir changer entre les données de JAMF et celles de GLPI
- Ajouter un composant IA pour faire des requêtes personnalisées à la base de données

4.2 Phase d'analyse de JAMF

JAMF est un agent d'inventaire remplissant les mêmes fonctions que GLPI mais avec une interface plus moderne et plus d'options. Il est spécialement conçu pour les Mac ce qui correspond parfaitement au besoin du parc étant donné que tout le monde travaille sur des Mac.

Afin de renseigner la machine sur JAMF, il faut l'inscrire, cela consiste à aller sur l'URL JAMF du serveur Polytechnique et rentrer ses identifiants JAMF. Après cela, JAMF s'occupe de récupérer toutes les informations concernant la machine et les met directement à disposition avec son API.

4.3 Analyse fonctionnelle

Fonctionnalité 1 : Modifier la structure de la base de données

En premier lieu, il faut que je rajoute des tables me permettant de récupérer les informations sur les ordinateurs et les logiciels de JAMF pour savoir s'ils sont vulnérables à certaines CVE. De plus, je stockais le nombre de machines affectées dans le champ de la

CVE elle-même, mais maintenant il ne m'est plus possible de faire ça car il faut distinguer les CVE touchant les machines JAMF des machines GLPI.

Surtout que certaines machines seront renseignées sur JAMF et GLPI en même temps ce qui faussera les données, j'ai donc restructuré la manière dont la base de données fonctionne en ajoutant des tables séparant les ordinateurs JAMF et GLPI.

Fonctionnalité 2 : Récupérer les informations de JAMF et les insérer dans la base.

Comme pour les autres logiciels, JAMF utilise des tokens pour s'authentifier lors des requêtes API, j'en ai donc créé un par moi-même pour me permettre d'interroger le serveur API. Par la suite, il faudra analyser les informations que le serveur me renvoie, les récupérer et les insérer dans la base de données.

Fonctionnalité 3 : Adapter la page web pour pouvoir changer entre les données de JAMF et celles de GLPI.

Mon API ne renvoyant que les informations concernant les ordinateurs de GLPI et leurs vulnérabilités, il faut ajouter la même fonctionnalité pour JAMF en différenciant chaque donnée pour ne pas afficher les mauvaises données.

L'utilisateur pourra choisir les données qu'il souhaite afficher en cliquant sur un bouton qui affichera soit les données de GLPI, soit les données de JAMF.

Fonctionnalité 4 : Ajouter un composant IA pour faire des requêtes personnalisées à la base de données.

L'utilisateur peut souhaiter voir certaines données uniquement formulables par une requête spécifique. L'ajout du composant IA permettra à l'utilisateur de formuler sa requête et de la traduire en langage SQL pour récupérer les informations depuis la base et les afficher dans le composant.

4.4 Analyse détaillée

Fonctionnalité 1 : Modifier la structure de la base de données

Ajout d'une nouvelle table pour stocker les ordinateurs de JAMF, ainsi que d'une table pour stocker les logiciels de ces ordinateurs et une table pour stocker les CVE concernant les systèmes d'exploitation et les logiciels des ordinateurs JAMF.

Fonctionnalité 2 : Récupérer les informations de JAMF et les insérer dans la base.

Création d'une fonction me permettant de créer un client JAMF pour m'authentifier lors des requêtes API.

Création des fonctions effectuant les requêtes à l'API JAMF pour en récupérer les données et les insérer dans la base de données.

Fonctionnalité 3 : Adapter la page web pour pouvoir changer entre les données de JAMF et celles de GLPI.

Mise à jour des données que mon API met à disposition pour renvoyer les ordinateurs JAMF et GLPI avec un tag spécifique pour me permettre de trier les données à l'affichage.

Fonctionnalité 4 : Ajouter un composant IA pour faire des requêtes personnalisées à la base de données.

Création d'un nouveau composant sur la page web permettant d'écrire et d'envoyer la requête au programme.

Traduction de la requête en langage SQL pour récupérer les informations de la base de données.

Affichage des informations renvoyées sur la page web.

4.5 Réalisation des missions

4.5.1 Modifier la structure de la base de données.

Afin de pouvoir insérer les données des ordinateurs et logiciels venant de JAMF, j'ai décidé de séparer la base de données en deux parties, une partie pour les informations de GLPI et l'autre pour les informations de JAMF.

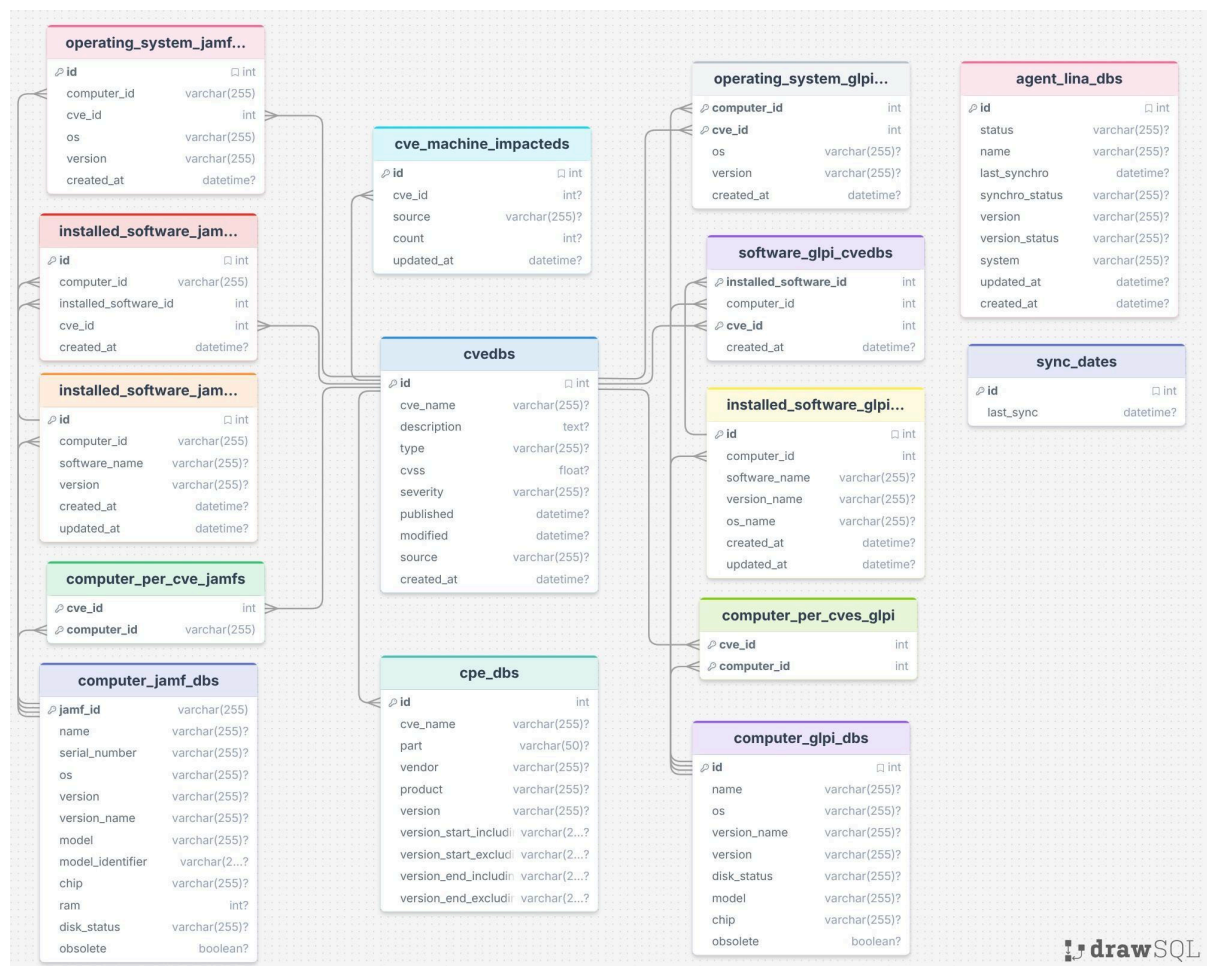


Figure 14 : Schéma relationnel adapté de la base de données

De cette manière, je me retrouve avec une architecture similaire à la précédente avec des tables stockant les informations des machines et des logiciels, des tables pour stocker les CVE affectant ces machines et logiciels.

Cependant, il ne m'était plus possible d'utiliser le champ des machines impactées dans ma table « cvedbs » car la même machine pouvait être renseignée dans JAMF et en même temps dans GLPI ce qui fausserait le nombre car elle serait présente deux fois dans ma base de données. Pour pallier ce problème, j'ai créé des tables intermédiaires « computer_per_cves » qui contiendront le nombre d'ordinateurs affectés par une CVE en fonction de leur provenance, JAMF ou GLPI.

4.5.2 Récupérer les informations de JAMF et les insérer dans la base.

Contrairement à l'API de GLPI, l'API de JAMF est très simple d'utilisation, il me suffit de faire une requête au serveur et celui-ci va me renvoyer toutes les informations en détail. Par exemple, si je souhaite récupérer les données d'un ordinateur, je vais pouvoir faire une requête en ajoutant certaines sections, notamment la section sur les disques qui me permet de tout récupérer sans passer par plusieurs appels différents pour reconstruire les données.

Après avoir récupéré les données, il m'a suffi de créer une fonction qui insère les informations récupérées, d'ajouter une vérification au cas où une information existerait déjà pour la mettre à jour si besoin et d'ajouter un champ à ma table me permettant de différencier les ordinateurs venant de JAMF de ceux venant de GLPI.

4.5.3 Adapter la page web pour pouvoir changer entre les données de JAMF et celles de GLPI.

Maintenant que les données sont stockées dans ma base, il suffit de simplement ajouter les objets à ce que mon API mettait déjà à disposition pour que ma page web puisse récupérer les informations.

Afin de pouvoir switcher rapidement entre les données de GLPI et de JAMF, j'ai décidé de créer un slider qui, quand il est cliqué, affiche les données concernant le logiciel qui est sélectionné.

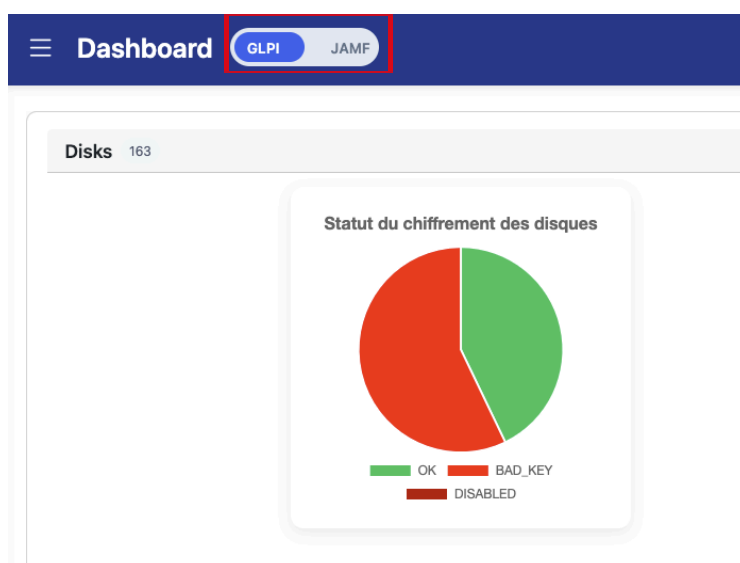


Figure 15 : Slider GLPI et JAMF

Cependant, les données renvoyées par l'API de JAMF et GLPI ne sont pas forcément les mêmes, par exemple pour le statut des disques de GLPI, l'information qu'un ordinateur a une mauvaise clé sera sous la forme « BAD_KEY » alors que JAMF sera sous la forme « Invalid Key ». Cela implique que mon graphique ne marchait pas initialement car il attendait les données sous la forme de GLPI pour se créer.

Trouver une solution ne fut pas très compliqué car ma page web fonctionnant en utilisant des composants, il suffisait de savoir quels composants l'utilisateur voulait et en fonction, afficher ces derniers. J'ai donc deux graphiques, un étant destiné à l'affichage des disques de GLPI, l'autre à l'affichage des disques de JAMF. De cette manière, la page web devient elle aussi flexible car si jamais un composant ou une autre source d'information venait à être ajouté, le développeur pourra créer un nouveau composant à part entière sans risque que les autres ne fonctionnent plus.

4.5.4 Ajouter un composant IA pour faire des requêtes personnalisées à la base de données.

Tout d'abord, il fallait réfléchir à comment introduire une intelligence artificielle dans mon programme car ce dernier ayant des données sensibles, il ne faudrait pas que ces dernières soient envoyées lors d'une requête à l'IA. Pour l'objectif de mon projet, il serait également mieux d'utiliser un modèle d'IA pouvant fonctionner en local et de ne pas s'appuyer sur un service d'intelligence artificielle déjà existant. Avec ces contraintes en tête, mon tuteur m'a conseillé d'utiliser une plateforme IA s'appelant Ollama. Ollama est un outil qui fonctionne en local, mettant à disposition des modèles IA à télécharger et à utiliser, notamment des LLM qui sont des modèles adaptés à mon besoin de traduire les requêtes en langage SQL. Lorsqu'un modèle est téléchargé, il est ensuite utilisable en interrogeant une API locale mise à disposition par Ollama. Cette dernière va permettre d'interroger et de récupérer les informations que l'IA aura traduites en langage SQL. Ainsi, mon programme pourra exécuter les requêtes SQL et retourner les données pour qu'elles soient affichées sur mon composant IA.

IA

Jamf_id	Name	Serial_number	Version_name	Model_identifieur	Ram	Obsolete	Os
15	mba-10828342	C02F11TPQ6LR	Sequoia	MacBookAir10,1	16384	0	macOS
11	mba-10834159	DV47R3FQTH	Tahoe	Mac14,2	16384	0	macOS

affiche moi tout les ordinateurs JAMF de la base de données

→

Figure 16 : Composant IA

Ensuite, je devais trouver une manière d'afficher les données dans le composant car selon la requête, le nombre et le nom des colonnes peuvent changer. Afin de résoudre ce problème, je me suis dit que la façon la plus efficace de mettre les données en forme serait de faire un tableau dynamique en fonction des champs de ma base de données. Par exemple, sur la figure ci-dessus, j'utilise les noms des champs de ma table d'ordinateur JAMF comme noms de colonnes pour afficher les données.

Conclusion

Mon stage dans l'équipe IDCS touchant à sa fin, il n'en reste pas moins une période très importante pour mon parcours académique et professionnel. J'ai pu consolider le savoir que j'ai acquis lors de mon précédent stage et le développer pour aboutir à un projet concret et fonctionnel.

Bilan professionnel

Étant familiarisé avec le Go et Svelte grâce à mon stage précédent, j'ai pu pousser mes compétences plus loin, que ce soit avec la création d'une page internet réactive et dynamique avec du drag and drop, mais aussi l'optimisation de mon programme pour minimiser le temps de calcul. Même si j'ai pu rencontrer des difficultés au cours de mon projet comme lorsque je devais récupérer des données avec les API de Lina ou GLPI car ces dernières ne proposaient pas de solution adaptée, cela m'a fait travailler ma capacité à trouver des solutions aux problèmes avec ce qui était à ma disposition.

Mon projet étant réalisé seul avec l'aide de l'équipe IDCS, j'ai gardé les bonnes approches que j'avais eues précédemment et pu en ajouter de nouvelles avec la création d'un dépôt Git pour faciliter le versionnage de mon code me permettant de tester des approches et de revenir en arrière si jamais elles ne fonctionnaient pas.

Le bon avancement de mon projet signifie qu'il sera amené à être déployé sur l'écran mural du bureau d'IDCS pour leur permettre de faire un suivi efficace.

Bilan personnel

Les cours suivis à l'IUT m'ont permis de m'adapter rapidement au travail qui m'était demandé notamment avec le système d'API sur lequel j'ai déjà pu travailler mais aussi l'intégration de Git à un projet.

Ce stage ralliant la programmation backend qui signifie l'infrastructure de mon application ainsi que sa manière de fonctionner et de récupérer les données, et le frontend pour tout ce qui est les pages web et l'interaction avec l'utilisateur me conforte dans mon choix de poursuivre une carrière en tant que développeur full stack.

Lexique

API : Application Programming Interface est une entité qui permet à deux composants logiciels de communiquer entre eux.

Composant logiciel : sont des éléments à part entière qui peuvent être incorporés ensemble pour créer une application.

Framework : est un ensemble de composants logiciels qui permet de créer les fondations d'un logiciel.

Librairie : en informatique cela représente un set de fonctions ou de classes déjà préparées par une autre personne dans un langage en particulier. Les librairies peuvent être importées dans le programme, et leurs fonctions utilisables directement, ce qui rajoute des options de programmation au langage initial.

CVE : Common Vulnerabilities and Exposures sont des vulnérabilités de sécurité concernant une liste de systèmes d'exploitation, de logiciel ou de matériels, elles décrivent les risques encourus et sont publiques.

CPE : Common Platform Enumeration permettent d'identifier l'élément qui contient une faille de sécurité, par qui cet élément a été fait et sur quelle version cette faille existe.

NVD : National Vulnerability Database est une base de données du gouvernement américain mettant à disposition les failles de sécurité des systèmes d'exploitation ou des logiciels pour permettre de vérifier si les machines sont affectées par ces failles.

LLM : Large Language Model est un type de modèle IA qui se spécialise dans la reconnaissance de texte et la génération de texte.

Endpoint API : est une URL spécifique à une API qui reçoit et envoie des requêtes pour faire communiquer les différentes parties d'un programme.

JSON : JavaScript Object Notation est un format d'échange de données permettant de créer des objets et de faire communiquer des navigateurs et des serveurs entre eux en envoyant ces objets.

Drag-and-Drop : est une technique qui consiste à sélectionner et à le déplacer d'un endroit à un autre sur un élément graphique.

Markdown : est un langage de balisage léger qui a pour but d'offrir une syntaxe facile à lire et à écrire, il est notamment utilisé par la communauté scientifique mais aussi dans la rédaction des fichiers README.md.

Logs : sont des fichiers générés par un programme qui permet de suivre l'activité du programme pour voir s'il se déroule sans problème d'exécution.

Table des figures

Figure 1 : Ecole Polytechnique	5
Figure 2 : Organigramme du CMLS	6
Figure 3 : Organigramme du CPHT	7
Figure 4 : Organigramme du CMAP	8
Figure 5 : Schéma relationnel de la base de données	16
Figure 6 : Capture d'écran de l'API	19
Figure 7 : Composant du statut des disques	20
Figure 8 : Composant des CVE	21
Figure 9 : Page web des détails de la CVE	22
Figure 10 : Composant des informations des ordinateurs	23
Figure 11 : Composant des agents Lina	24
Figure 12 : Composant des détails des CVE	25
Figure 13 : Affichage de la liste des composants	26
Figure 14 : Schéma relationnel adapté de la base de données	31
Figure 15 : Slider GLPI et JAMF	33
Figure 16 : Composant IA	34

Annexe

1. Vue d'ensemble de la page web

